SRE Watchdog

AI-Powered Intelligent Observability & Event Watchdog

MVP Presentation - June 2026

The Problem

- SRE teams drown in log noise - thousands of entries per minute
- Manual threshold monitoring misses subtle degradation patterns
- Alert fatigue from duplicate/noisy notifications
- No intelligent context in alerts - just raw numbers
- Need: AI-powered anomaly detection with actionable summaries

The Solution: SRE Watchdog

- Python 3.13 FastAPI application with AI-powered log analysis
- Two-gate detection: statistical pre-filter + AWS Bedrock AI
- Automatic webhook alerts with severity classification
- Real-time dashboard with Chart.js visualizations
- Rate limiting, input validation, and security scanning
- 95.5% test coverage with CI/CD pipeline
- Dockerized for portable deployment

Architecture Overview

- Gate 1: APScheduler tick - SQL-level COUNT for error rate
- Gate 2: FastAPI BackgroundTask - Bedrock Converse API (Claude Sonnet 4.6)
- Alert Service: Severity mapping + webhook dispatch with retry
- Cooldown: Prevents alert fatigue (15-min suppression window)
- Rate Limiting: 60/min ingest, 10/min analyze (slowapi)
- Dashboard: Jinja2 SSR + Chart.js + 60s auto-refresh
- Storage: SQLite WAL mode for concurrent read/write

Tech Stack

- Language: Python 3.13 (Docker) / 3.11+ (local)
- Framework: FastAPI + Uvicorn + slowapi (rate limiting)
- AI: AWS Bedrock (Claude Sonnet 4.6 via Converse API)
- Database: SQLite with WAL mode (SQLAlchemy ORM)
- Scheduling: APScheduler (BackgroundScheduler)
- Dashboard: Jinja2 + Chart.js (single aggregation query)
- Quality: pytest (95.5%), flake8, mypy, bandit, GitHub Actions CI

Key Features Delivered

- 10,000 synthetic logs across 5 services (24-hour simulation)
- 3 seeded anomaly windows: sharp spike, sustained degradation, cascade
- AI-generated anomaly summaries with severity scoring (0.0-1.0)
- Webhook alerts with 4-band severity (LOW/MEDIUM/HIGH/CRITICAL)
- Cooldown suppression + credential failure auto-purge on restart
- On-demand analysis via POST /analyze + async job polling
- 11 API endpoints with OpenAPI/Swagger documentation

Dashboard: Metrics & Error Rate Chart

Metrics bar showing logs ingested, anomalies, alerts, and failures. Chart.js line chart with error rate per service over 24 hours.

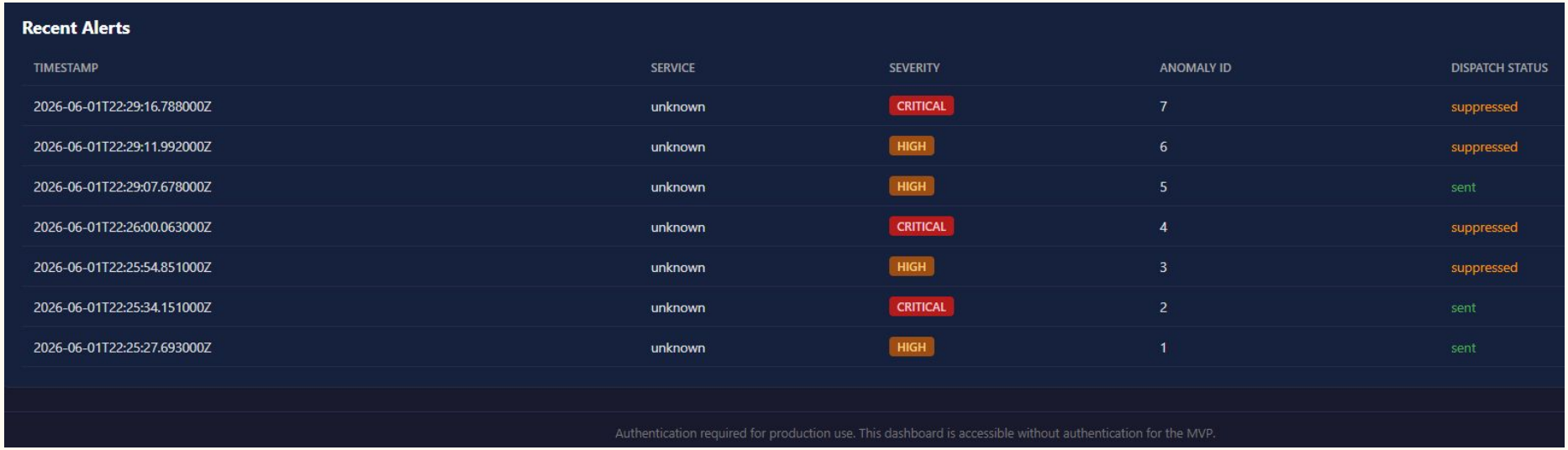
Dashboard: Anomaly Detection Results

Recent Anomalies

SERVICE	WINDOW	SCORE	STATUS	SEVERITY	AI SUMMARY
database-proxy	2026-06-01T21:29:00 – 2026-06-01T22:29:00	0.92	suppressed	CRITICAL	Critical multi-vector failure detected within the observation window: the primary database disk is full and writes are halted, a connection pool corruption triggered an emergency restart, and the connection pool has been exhausted twice alongside multiple re-initializations indicating instability. Compounding issues include a deadlock on the transactions table, repeated lock wait timeouts on the orders table, and elevated read replica lag of 2500ms appearing multiple times, suggesting replication is falling behind likely due to the write halt. The convergence of write unavailability, connection layer instability, locking contention, and replication degradation constitutes a cascading failure requiring immediate escalation and intervention.
auth-service	2026-06-01T21:29:00 – 2026-06-01T22:29:00	0.82	suppressed	HIGH	The auth-service is experiencing a significant multi-vector failure pattern within the observation window, with a 12% error rate driven by three distinct failure types: complete session store unreachability causing total auth failure, repeated JWT malformed payload decode errors, an LDAP bind timeout, and a token signature mismatch. The session store being completely unreachable is the most critical signal, as it implies all authentication flows depending on session persistence are failing, not just degraded. The combination of infrastructure-level failure (session store, LDAP) alongside JWT integrity errors suggests either a cascading dependency issue or an underlying infrastructure disruption that warrants immediate escalation.
api-gateway	2026-06-01T21:29:00 – 2026-06-01T22:29:00	0.74	alerted	HIGH	The api-gateway is experiencing multi-dimensional degradation with a 10.7% error rate driven by repeated connection timeouts to auth-service (3 occurrences), connection pool exhaustion (2 dropped requests), an open circuit breaker on payment-service, and 3 upstream response time threshold breaches. The combination of a tripped circuit breaker on a critical downstream service (payment-service), repeated auth-service timeouts, and connection pool pressure suggests cascading upstream failures rather than isolated transient errors. Immediate investigation into auth-service and payment-service health is warranted, along with connection pool sizing and timeout configuration review.
database-proxy	2026-06-01T21:25:48 – 2026-06-01T22:25:48	0.91	suppressed	CRITICAL	A critical combination of failures is present in this time window: the primary database disk is full and writes are halted, a connection pool corruption triggered an emergency restart, and deadlock/lock wait timeout events on core tables (transactions, orders) indicate active contention. Compounding these are repeated connection pool exhaustions (all 20 connections consumed), elevated read replica lag spikes to 2500ms, and an unusually high number of connection pool re-initializations (7 occurrences), strongly suggesting cascading instability. Immediate action is required to clear disk capacity on the primary, investigate the connection pool corruption root cause, and resolve table-level locking issues before full service failure occurs.
auth-service	2026-06-01T21:25:48 – 2026-06-01T22:25:48	0.74	suppressed	HIGH	The auth-service is exhibiting multiple concurrent failure modes within the observation window, including complete session store unavailability (2 occurrences), JWT decode errors with malformed payloads (3 occurrences), an LDAP bind timeout, a token signature mismatch, and elevated session store latency at 450ms. The combination of session store unreachability and JWT integrity failures suggests both infrastructure-level degradation and possible token pipeline corruption occurring simultaneously. With a 11.2% error rate driven by distinct, overlapping failure categories rather than a single isolated fault, this warrants prompt escalation and investigation into session store connectivity and token issuance integrity.
database-proxy	2026-06-01T21:25:20 – 2026-06-01T22:25:20	0.92	alerted	CRITICAL	The database-proxy is experiencing a critical multi-vector failure within the observation window. A primary database disk-full event has halted all writes, a connection pool corruption required emergency restart, and a deadlock was detected on the transactions table — compounding an already strained system showing repeated connection pool exhaustion, elevated read replica lag (2500ms spikes), and lock wait timeouts on the orders table. The convergence of storage failure, connection pool instability, and locking contention indicates a cascading breakdown requiring immediate escalation and intervention.
auth-service	2026-06-01T21:25:20 – 2026-06-01T22:25:20	0.74	alerted	HIGH	The auth-service is exhibiting multiple concurrent failure modes within the observation window, including complete session store unavailability (2 occurrences), JWT decode errors with malformed payload segments (3 occurrences), an LDAP bind timeout, a token signature mismatch, and elevated session store latency at 450ms. The combination of session store unreachability and JWT integrity failures suggests both infrastructure-level instability and possible token pipeline corruption, which together pose significant risk of widespread authentication failure. Immediate investigation into the session store health and the source of malformed JWT payloads is warranted.

Recent anomalies table showing service, time window, AI score, lifecycle status, severity badge, and AI-generated summary.

Dashboard: Alert Dispatch Log

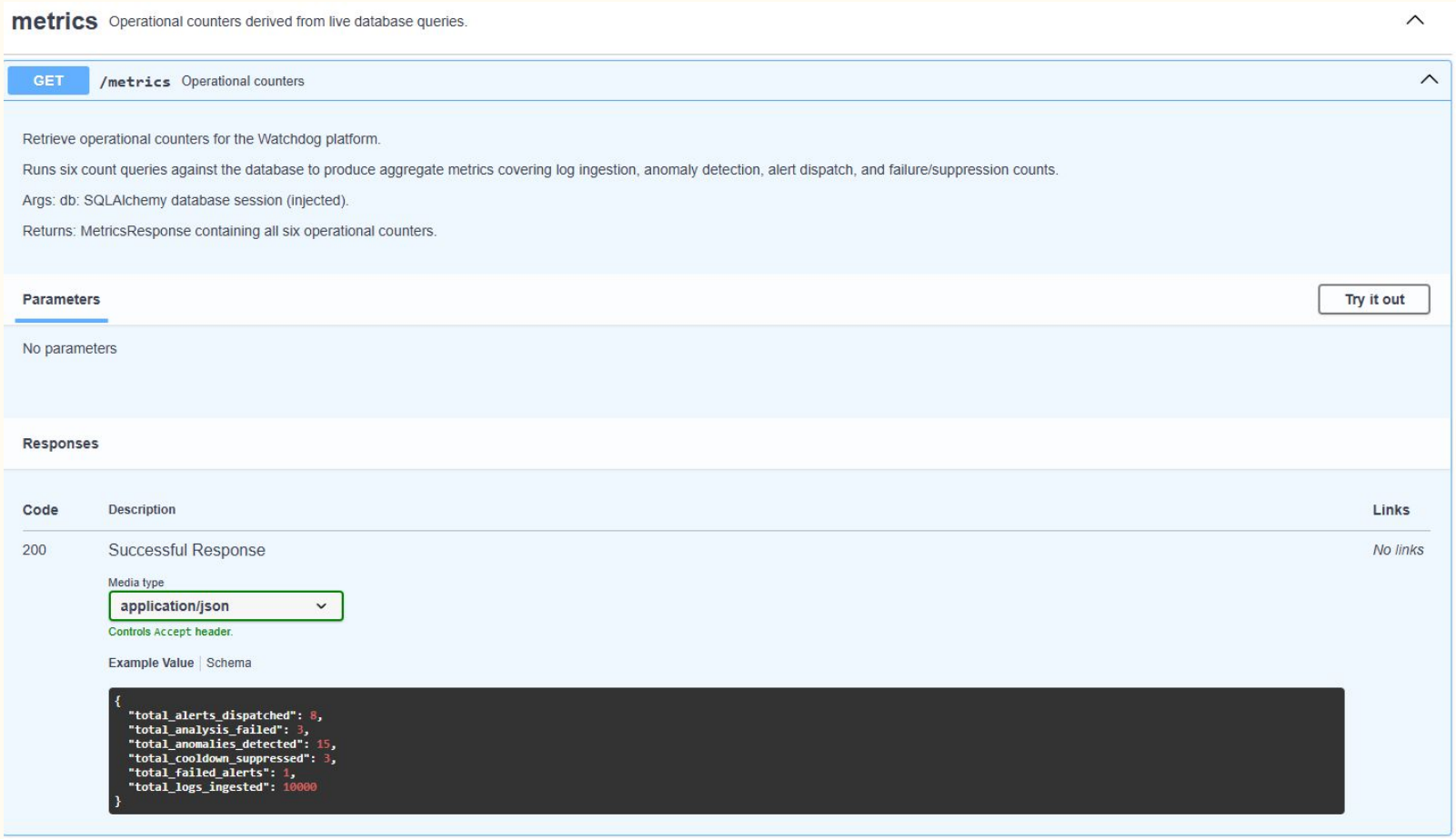


Recent alerts table showing timestamp, service, severity, anomaly ID, and dispatch status (sent/suppressed/failed).

Dashboard: On-Demand Analysis

Run Analysis button triggering Bedrock AI analysis across all services. Shows loading state and refreshes results on completion.

OpenAPI/Swagger Documentation



Interactive API docs at /docs with tag groups, request examples, response schemas, and error code documentation.

Detection Pipeline in Action

- 1. APScheduler tick fires every 60 seconds
- 2. Gate 1: SQL COUNT per service (no ORM objects loaded)
- 3. If error_rate $>$ 10%: create AnomalyWindow (pending_analysis)
- 4. Gate 2: BackgroundTask invokes Bedrock with log context
- 5. Bedrock returns anomaly_score (0.0-1.0) + AI summary
- 6. If score $>=$ 0.5 and no cooldown: dispatch webhook alert
- 7. Full lifecycle persisted for audit trail

AWS Bedrock AI Integration

- Model: Claude Sonnet 4.6 (us.anthropic.claude-sonnet-4-6)
- Prompt: Service context + error rate + log messages (capped at 50)
- Response: JSON with anomaly_score + plain-text summary
- Parser: Handles both raw JSON and markdown-fenced responses
- Retry: Exponential backoff (1s, 2s, 4s) for transient errors
- Health caching: /health reports last-known Bedrock status
- Cost: ~$0.006 per inference call ($1-18/month typical)

Testing & Quality Assurance

- 127 tests (71 unit + 54 integration + 1 property-based + 1 live Bedrock)
- 95.5% code coverage with branch coverage enabled
- 16 modules at 100% coverage (all critical paths)
- Property-based test: Hypothesis round-trip validation
- Security: bandit SAST scanning (0 High issues)
- Linting: flake8 (zero errors) + mypy (type checking)
- CI/CD: GitHub Actions matrix (Python 3.11/3.12/3.13)

Security & Performance

- Rate limiting: 60/min ingest, 10/min analyze (HTTP 429)
- Input validation: service name restricted to 5 known services
- SQL injection: parameterized queries via SQLAlchemy ORM
- Performance: Single GROUP BY query for dashboard (was 240)
- Performance: SQL COUNT for Gate 1 (no ORM objects in memory)
- Performance: OUTER JOIN for alerts (was N+1 queries)
- Startup cleanup: auto-purge credential-failure records

Challenges & Solutions

- Model ID iteration: 5 attempts to find working Bedrock model
- Response parsing: Claude wraps JSON in markdown fences
- Circular FK: use_alter=True resolves SQLAlchemy DROP warnings
- BackgroundTask testing: Option 1 (direct) + Option 2 (TestClient exit)
- Credential expiry: auto-purge + structured error in dashboard
- Dashboard N+1: OUTER JOIN for service names in alert list

Docker & Deployment

- Dockerfile: python:3.13-slim with health check
- Single command: docker run -p 8000:8000 sre-watchdog:latest
- AWS credentials via environment variables
- Dashboard at http://localhost:8000/dashboard
- Swagger UI at http://localhost:8000/docs
- Production path: App Runner or ECS/Fargate + RDS PostgreSQL

Production Roadmap

- Authentication: OAuth2/OIDC for dashboard access
- Deployment: AWS App Runner or ECS/Fargate
- Database: Migrate to RDS PostgreSQL (config-only change)
- Semantic analysis: S3 Vectors for log embeddings
- Cursor-based pagination for high-volume log stores
- OpenTelemetry: Distributed tracing integration
- Prometheus: /metrics in standard exposition format

Project Statistics

- Total time: ~14.5 hours (within 16-hour window)
- Code generation (all tasks): 37 minutes
- Source files: 25 app modules + 16 test files
- Test coverage: 95.5% (127 tests passing)
- Documentation: 10 markdown files + spec deviations
- API: 11 endpoints with OpenAPI examples
- Security: 0 High bandit findings, rate-limited

Live Demo

- 1. docker run -p 8000:8000 sre-watchdog:latest
- 2. python generate_logs.py (10K logs, 3 anomaly windows)
- 3. Open http://localhost:8000/dashboard
- 4. Click 'Run Analysis' - watch Bedrock AI in action
- 5. Observe: anomaly scores, AI summaries, alert dispatch
- 6. Check /docs (Swagger), /health, /metrics

Thank You

SRE Watchdog - AI-Powered Observability
GitHub: github.com/archaws25-git/sre-watchdog
Questions?